

KUBERNETES-TASK-05

1. Create a namespace dev-environment and apply a resource-based quota that restricts the number of pods to 3 and services to 2.

Step 1: Create the namespace:

```
ubuntu@k8s-master:~$ kubectl create namespace dev-environment
namespace/dev-environment created
ubuntu@k8s-master:~$ kubectl get namespaces
NAME                STATUS    AGE
default              Active    7d2h
dev-environment      Active    25s
kube-flannel         Active    7d2h
kube-node-lease      Active    7d2h
kube-public          Active    7d2h
kube-system          Active    7d2h
```

Step 2: Create ResourceQuota YAML:

Create a file:

vi dev-quota.yaml

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev-environment
spec:
  hard:
    pods: "3"
    services: "2"
```

Step 3: Apply the quota

```
ubuntu@k8s-master:~$ vi dev-quota.yaml
ubuntu@k8s-master:~$ kubectl apply -f dev-quota.yaml
resourcequota/dev-quota created
```

Step 4: Verify

```
ubuntu@k8s-master:~$ kubectl get resourcequota -n dev-environment
NAME          AGE    REQUEST              LIMIT
dev-quota     60s    pods: 0/3, services: 0/2
ubuntu@k8s-master:~$ kubectl describe resourcequota dev-quota -n dev-environment
Name:         dev-quota
Namespace:    dev-environment
Resource      Used  Hard
-----
pods          0     3
services      0     2
ubuntu@k8s-master:~$
```

2. Create a pod in the prod-environment namespace with 0.2 CPU and 200Mi memory requests, and 0.5 CPU and 500Mi memory limits.

Step 1: Create namespace

kubectl create namespace prod-environment

```
ubuntu@k8s-master:~$ kubectl create namespace prod-environment
namespace/prod-environment created
ubuntu@k8s-master:~$
```

Step 2: Create Pod YAML

Create:

vi prod-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: prod-pod
  namespace: prod-environment
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    resources:
      requests:
        cpu: "200m"
        memory: "200Mi"
      limits:
        cpu: "500m"
        memory: "500Mi"
```

Step 3: Create the Pod & Check Pod:

```
ubuntu@k8s-master:~$ vi prod-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f prod-pod.yaml
pod/prod-pod created
ubuntu@k8s-master:~$ kubectl get pods
No resources found in default namespace.
ubuntu@k8s-master:~$ kubectl get pod
No resources found in default namespace.
ubuntu@k8s-master:~$ kubectl get pods -n prod-environment
NAME        READY   STATUS    RESTARTS   AGE
prod-pod    1/1     Running   0           71s
ubuntu@k8s-master:~$
```

Step 5: Check resources:

```

ubuntu@k8s-master:~$ kubectl describe pod prod-pod -n prod-environment
Name:          prod-pod
Namespace:     prod-environment
Priority:       0
Service Account: default
Node:          worker01/172.31.4.11
Start Time:    Tue, 29 Sep 2026 10:21:16 +0000
Labels:        <none>
Annotations:   <none>
Status:        Running
IP:            10.244.1.73
IPs:
  IP: 10.244.1.73
Containers:
  nginx:
    Container ID:   containerd://3a41f8e49f751bc1195d6da4f5d8edf2d098077581c7976c05f23ede85f9783b
    Image:          nginx:1.27
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Tue, 29 Sep 2026 10:21:17 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:           500m
      memory:        500Mi
    Requests:
      cpu:           200m
      memory:        200Mi
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-8gft4 (ro)
Conditions:

```

What is Request?

Request = resource Kubernetes reserves for the container.

Example:

CPU request = 200m

Kubernetes scheduler looks for a node having enough available CPU to satisfy that request.

What is Limit?

Limit = maximum resource the container can use.

Here:

CPU maximum = 500m

Memory maximum = 500Mi

Easy example

Think about a hotel room:

Request = room reserved for you

Limit = maximum room capacity available to you

3. In the staging-environment namespace, set a LimitRange that assigns default CPU and memory limits (300m CPU, 600Mi memory) and applies a minimum and maximum CPU.

Step 1: Create namespace

kubectl create namespace staging-environment

```
ubuntu@k8s-master:~$ kubectl create namespace staging-environment
namespace/staging-environment created
ubuntu@k8s-master:~$ kubectl get namespaces
NAME                STATUS    AGE
default              Active    7d3h
dev-environment      Active    31m
kube-flannel         Active    7d3h
kube-node-lease      Active    7d3h
kube-public           Active    7d3h
kube-system           Active    7d3h
prod-environment     Active    16m
staging-environment  Active    63s
```

Step 2: Create LimitRange:

Create:

vi staging-limitrange.yaml

```
apiVersion: v1
kind: LimitRange
metadata:
  name: staging-limits
  namespace: staging-environment
spec:
  limits:
    - type: Container
      default:
        cpu: "300m"
        memory: "600Mi"
      min:
        cpu: "100m"
      max:
        cpu: "500m"
```

```
ubuntu@k8s-master:~$ vi staging-limitrange.yaml
ubuntu@k8s-master:~$ kubectl apply -f staging-limitrange.yaml
limitrange/staging-limits created
```

Step 4: Verify:

```
ubuntu@k8s-master:~$ kubectl get limitrange -n staging-environment
NAME                CREATED AT
staging-limits      2026-09-29T10:40:16Z
ubuntu@k8s-master:~$

ubuntu@k8s-master:~$ kubectl describe limitrange staging-limits -n staging-environment
Name:                staging-limits
Namespace:            staging-environment
Type                 Resource  Min    Max    Default Request  Default Limit  Max Limit/Request Ratio
-----
Container            cpu       100m   500m   300m             300m           -
Container            memory    -      -      600Mi            600Mi          -
ubuntu@k8s-master:~$
```

Now test the LimitRange

This is the important part.

Create a pod **without** specifying resources.

```

apiVersion: v1
kind: Pod
metadata:
  name: staging-pod
  namespace: staging-environment
spec:
  containers:
    - name: nginx
      image: nginx:1.27
~
~

```

Notice:

resources:

is completely missing.

Apply it:

```

ubuntu@k8s-master:~$ vi staging-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f staging-pod.yaml
pod/staging-pod created
ubuntu@k8s-master:~$

```

Check:

```

ubuntu@k8s-master:~$ kubectl get pod staging-pod -n staging-environment
NAME          READY   STATUS    RESTARTS   AGE
staging-pod   1/1     Running   0           88s
ubuntu@k8s-master:~$ kubectl describe pod staging-pod -n staging-environment
Name:          staging-pod
Namespace:     staging-environment
Priority:       0
Service Account: default
Node:          worker02/172.31.4.12
Start Time:    Tue, 29 Sep 2026 10:46:40 +0000
Labels:        <none>
Annotations:   kubernetes.io/limit-ranger: LimitRanger plugin set: cpu, memory request for container nginx; cpu, memory limit for container nginx
Status:        Running
IP:            10.244.2.71
IPs:
  IP: 10.244.2.71
Containers:
  nginx:
    Container ID:   containerd://ce17cce5848c71b6b3feb2a4335d2ed4e83bfb35c45bc971balcd32b62486d54
    Image:          nginx:1.27
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Tue, 29 Sep 2026 10:46:41 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:          300m
      memory:       600Mi
    Requests:
      cpu:          300m
      memory:       600Mi
    Environment:    <none>

```

Look at the container resources.

Kubernetes should automatically apply:

CPU limit = 300m

Memory limit = 600Mi

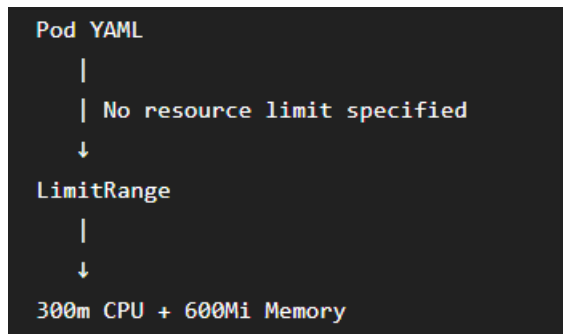
Why?

Because we configured:

default:

cpu: "300m"

memory: "600Mi"



4. Create a pod and a NodePort service in the default namespace, then create another pod in the test namespace and communicate between them using Service DNS.

Step 1: Create nginx Pod in default namespace

Create:

vi default-nginx.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: default-nginx
  labels:
    app: default-nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

Apply:

```
ubuntu@k8s-master:~$ kubectl apply -f default-nginx.yaml
pod/default-nginx created
ubuntu@k8s-master:~$
```

Check:

```
ubuntu@k8s-master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
default-nginx 1/1     Running   0           23s
ubuntu@k8s-master:~$
```

Step 2: Create NodePort Service

Create:

vi nginx-service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: default-nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
~
ubuntu@k8s-master:~$ kubectl apply -f nginx-service.yaml
service/nginx-service created
ubuntu@k8s-master:~$

```

Step 3: Verify Service

```

ubuntu@k8s-master:~$ kubectl get svc

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	76m
nginx-service	NodePort	10.107.191.109	<none>	80:30080/TCP	13m

```

ubuntu@k8s-master:~$

```

Step 4: Check Service endpoints

```

ubuntu@k8s-master:~$ kubectl get endpoints nginx-service

```

NAME	ENDPOINTS	AGE
nginx-service	10.244.1.74:80	17m

```

ubuntu@k8s-master:~$

```

Step 5: Create test namespace

```

ubuntu@k8s-master:~$ kubectl create namespace test
namespace/test created

```

Step 6: Create test Pod & apply

```

ubuntu@k8s-master:~$ vi test-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f test-pod.yaml
pod/test-pod created

```

```

apiVersion: v1
kind: Pod
metadata:
  name: test-pod
  namespace: test
spec:
  containers:
    - name: test-container
      image: busybox:1.36
      command:
        - sleep
        - "3600"
~

```

Verify:

```

ubuntu@k8s-master:~$ kubectl get pods -n test

```

NAME	READY	STATUS	RESTARTS	AGE
test-pod	1/1	Running	0	6s

Step 7: Enter the test Pod & Step 8: Test Service DNS

```
ubuntu@k8s-master:~$ kubectl exec -it test-pod -n test -- sh
/ #
/ # wget -qO- http://nginx-service.default.svc.cluster.local
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
```

5. Apply a LimitRange with a max limit/request ratio of 2 for memory in the performance-environment namespace, and test by creating a pod with mismatched resource requests and limits.

Step 1: Create namespace

```
ubuntu@k8s-master:~$ kubectl create namespace performance-environment
namespace/performance-environment created
ubuntu@k8s-master:~$
```

Step 2: Create LimitRange

```
ubuntu@k8s-master:~$ vi performance-limitrange.yaml
ubuntu@k8s-master:~$
```

```
apiVersion: v1
kind: LimitRange
metadata:
  name: performance-limits
  namespace: performance-environment
spec:
  limits:
    - type: Container
      maxLimitRequestRatio:
        memory: "2"
```

Apply:

```
ubuntu@k8s-master:~$ kubectl apply -f performance-limitrange.yaml
limitrange/performance-limits created
ubuntu@k8s-master:~$
```

Verify:

```
ubuntu@k8s-master:~$ kubectl describe limitrange performance-limits -n performance-environment
Name:         performance-limits
Namespace:    performance-environment
Type          Resource  Min  Max  Default Request  Default Limit  Max Limit/Request Ratio
-----
Container     memory    -    -    -                 -                2
ubuntu@k8s-master:~$
```

Understand ratio first

Suppose:

Memory request = 200Mi

Memory limit = 400Mi

Calculate:

$$400 / 200 = 2$$

So this is allowed.

But:

Memory request = 200Mi

Memory limit = 500Mi

Calculate:

$$500 / 200 = 2.5$$

Our maximum is:

2

Therefore:

$$2.5 > 2$$

The Pod should be rejected.

Step 3: Create a valid Pod

Create:

vi performance-valid.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: performance-valid
  namespace: performance-environment
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      resources:
        requests:
          memory: "200Mi"
        limits:
          memory: "400Mi"

ubuntu@k8s-master:~$ vi performance-valid.yaml
ubuntu@k8s-master:~$ kubectl apply -f performance-valid.yaml
pod/performance-valid created
ubuntu@k8s-master:~$

ubuntu@k8s-master:~$ kubectl get pods -n performance-environment
NAME                READY   STATUS    RESTARTS   AGE
performance-valid   1/1     Running   0           50s
ubuntu@k8s-master:~$
```

It should be created successfully.

Because:

$$400\text{Mi} / 200\text{Mi} = 2$$

Step 4: Create an invalid Pod

Create:

vi performance-invalid.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: performance-invalid
  namespace: performance-environment
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      resources:
        requests:
          memory: "200Mi"
        limits:
          memory: "500Mi"
```

Apply:

```
ubuntu@k8s-master:~$ kubectl apply -f performance-invalid.yaml
Error from server (Forbidden): error when creating "performance-invalid.yaml": pods "performance-invalid" is forbidden: memory max limit ratio is 2.500000
ubuntu@k8s-master:~$
```

Kubernetes should reject it because:

Request = 200Mi

Limit = 500Mi

$500 / 200 = 2.5$

But our LimitRange says:

Maximum ratio = 2

Therefore:

$2.5 > 2$